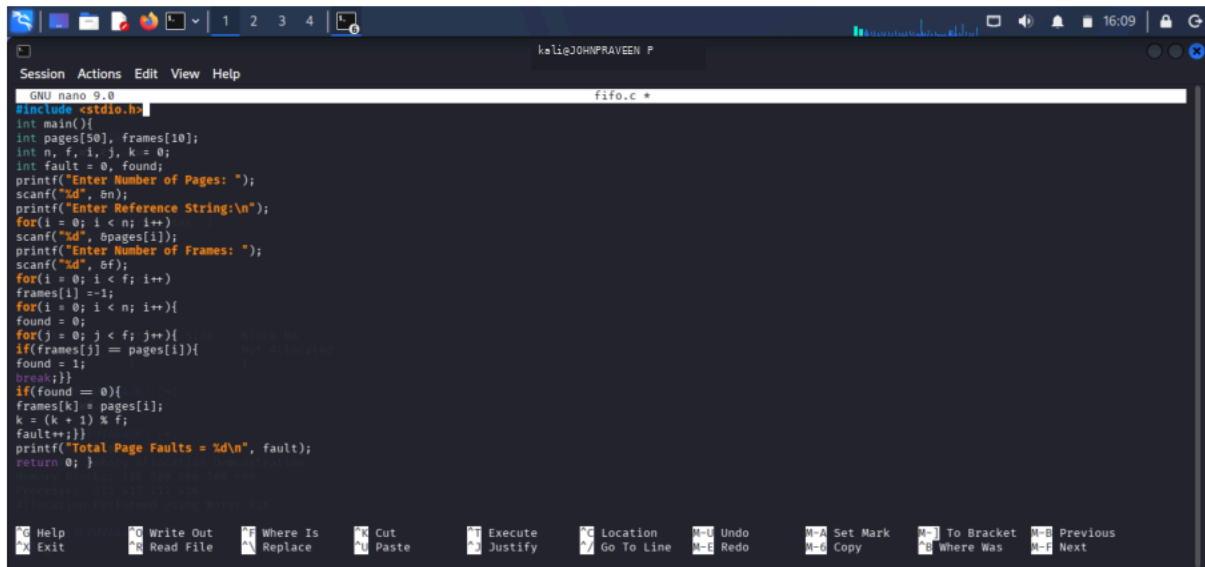
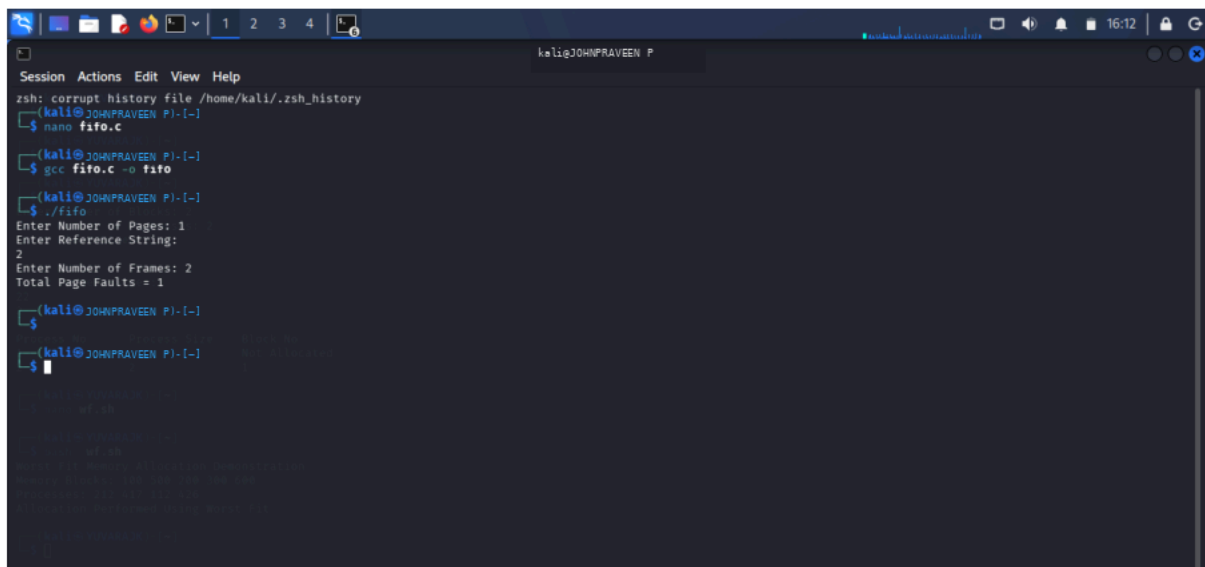


C PROGRAMMING : FIFO



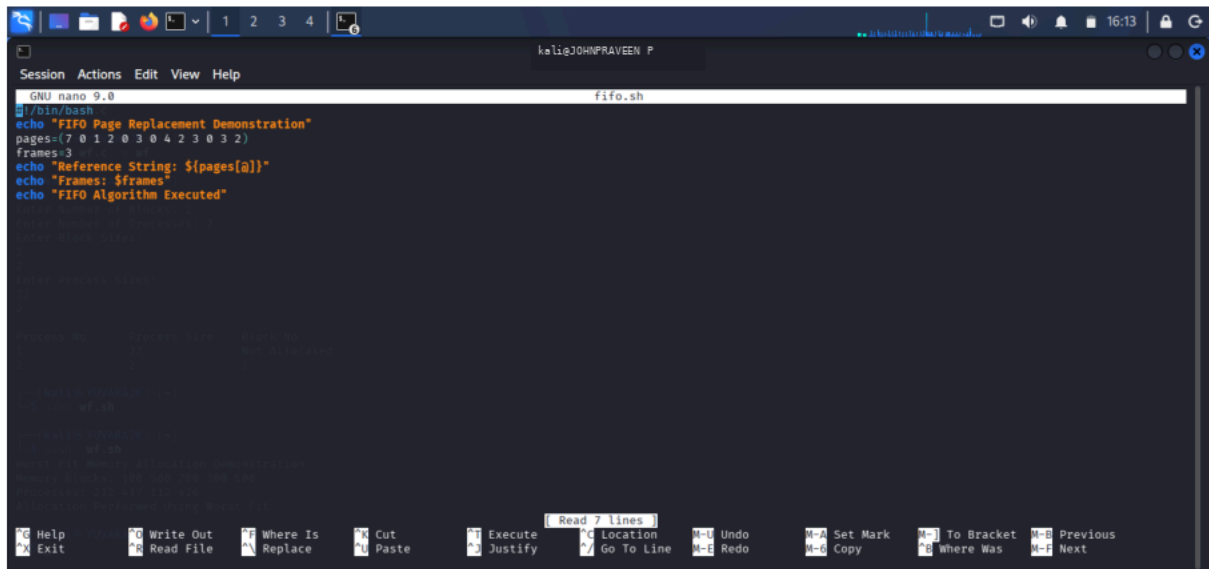
```
GNU nano 9.0 fifo.c
#include <stdio.h>
int main(){
    int pages[50], frames[10];
    int n, f, i, j, k = 0;
    int fault = 0, found;
    printf("Enter Number of Pages: ");
    scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &pages[i]);
        printf("Enter Number of Frames: ");
        scanf("%d", &f);
        for(i = 0; i < f; i++){
            frames[i] = -1;
        }
        for(i = 0; i < n; i++){
            found = 0;
            for(j = 0; j < f; j++){
                if(frames[j] == pages[i]){
                    found = 1;
                    break;
                }
            }
            if(found == 0){
                frames[k] = pages[i];
                k = (k + 1) % f;
                fault++;
            }
        }
        printf("Total Page Faults = %d\n", fault);
        return 0;
    }
```

OUTPUT :



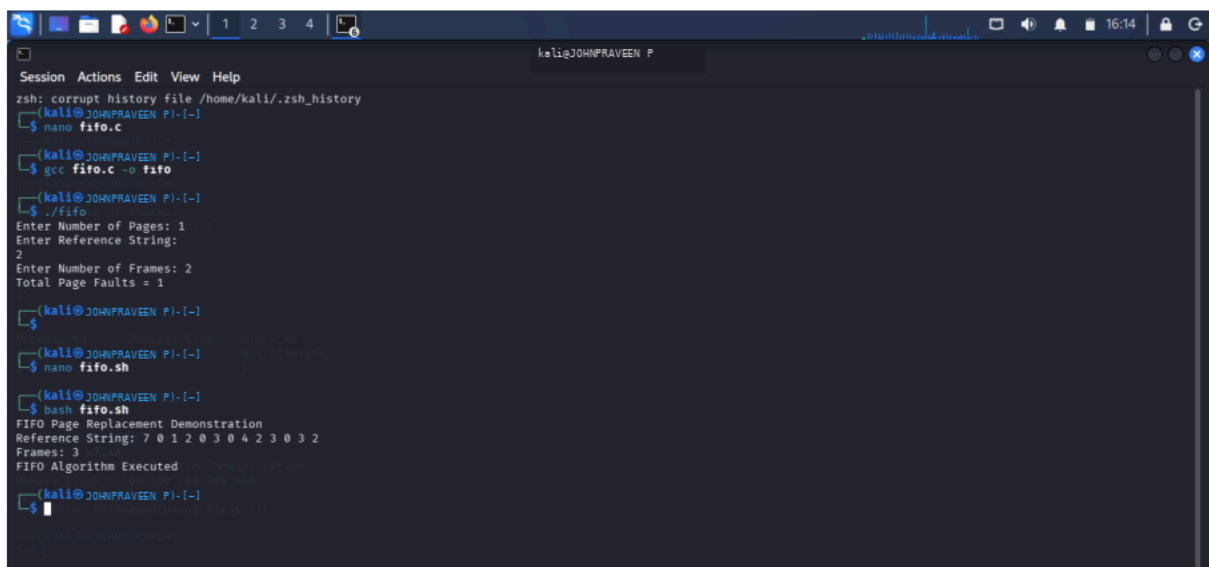
```
zsh: corrupt history file /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~$ nano fifo.c
kali@JOHNPRAVEEN P:~$ gcc fifo.c -o fifo
kali@JOHNPRAVEEN P:~$ ./fifo
Enter Number of Pages: 1
Enter Reference String:
2
Enter Number of Frames: 2
Total Page Faults = 1
kali@JOHNPRAVEEN P:~$
```

SHELL PROGRAMMING : FIFO



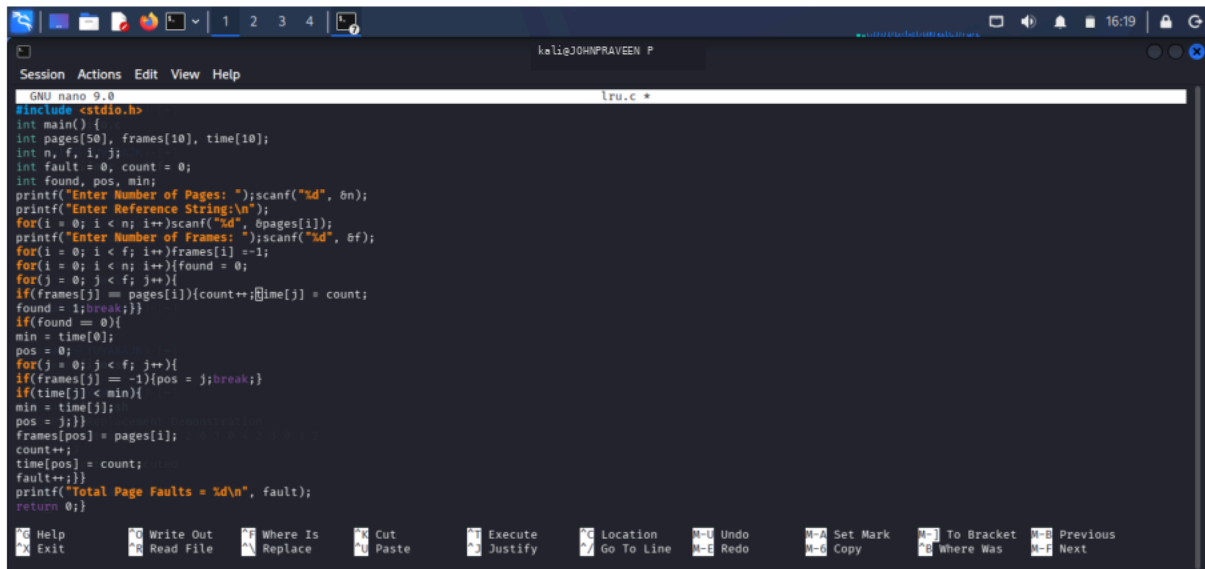
```
GNU nano 9.0 fifo.sh
#!/bin/bash
echo "FIFO Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
frames=3
echo "Reference String: ${pages[@]}"
echo "Frames: $frames"
echo "FIFO Algorithm Executed"
```

OUTPUT :



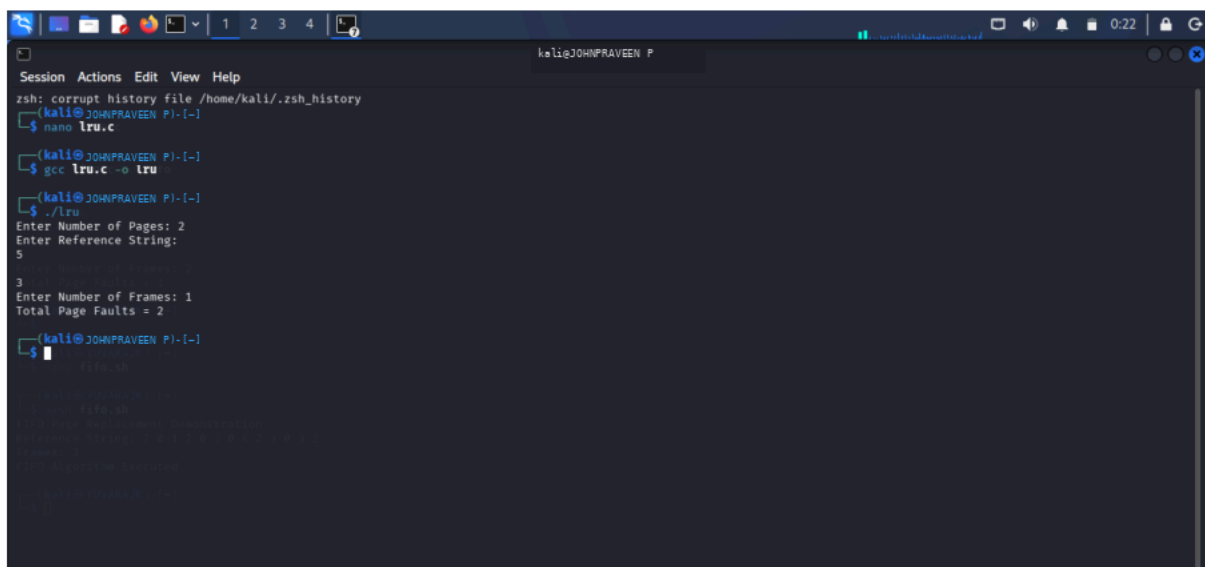
```
zsh: corrupt history file /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~$ nano fifo.c
kali@JOHNPRAVEEN P:~$ gcc fifo.c -o fifo
kali@JOHNPRAVEEN P:~$ ./fifo
Enter Number of Pages: 1
Enter Reference String:
2
Enter Number of Frames: 2
Total Page Faults = 1
kali@JOHNPRAVEEN P:~$ nano fifo.sh
kali@JOHNPRAVEEN P:~$ bash fifo.sh
FIFO Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Frames: 3
FIFO Algorithm Executed
kali@JOHNPRAVEEN P:~$
```

C PROGRAMMING :LRU



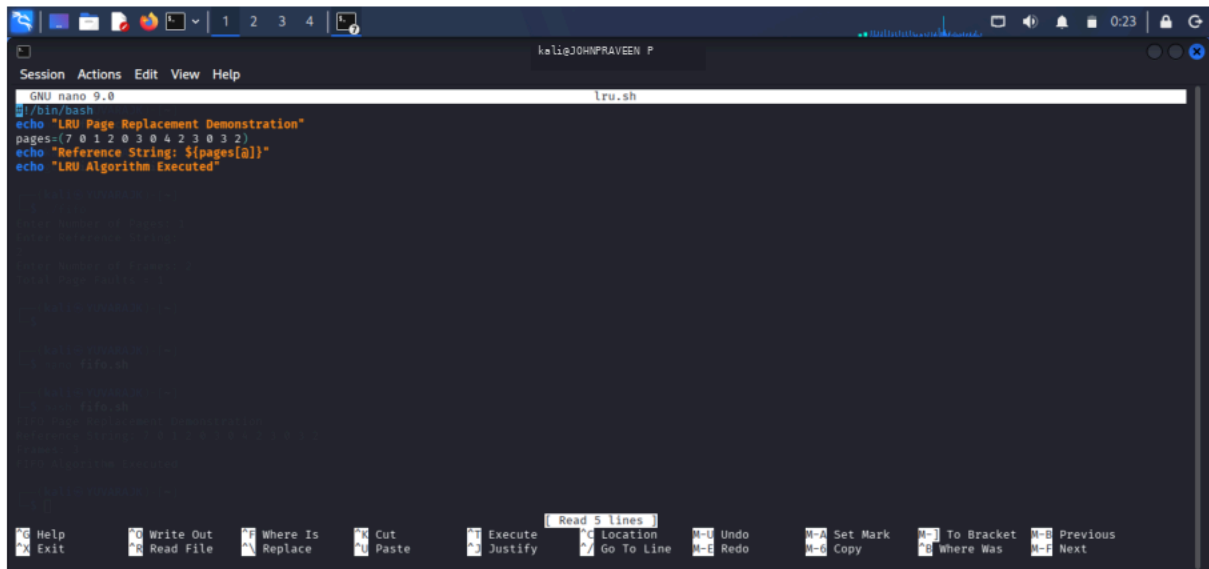
```
GNU nano 9.0 lru.c *
#include <stdio.h>
int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j;
    int fault = 0, count = 0;
    int found, pos, min;
    printf("Enter Number of Pages: ");scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++)scanf("%d", &pages[i]);
    printf("Enter Number of Frames: ");scanf("%d", &f);
    for(i = 0; i < f; i++)frames[i] = -1;
    for(i = 0; i < n; i++){found = 0;
    for(j = 0; j < f; j++){
        if(frames[j] == pages[i]){count++;time[j] = count;
        found = 1;break;}}
    if(found == 0){
        min = time[0];
        pos = 0;
        for(j = 0; j < f; j++){
            if(frames[j] == -1){pos = j;break;}
            if(time[j] < min){
                min = time[j];
                pos = j;}}
        frames[pos] = pages[i];
        count++;
        time[pos] = count;
        fault++;}}
    printf("Total Page Faults = %d\n", fault);
    return 0;
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHNPRAVEEN P)-[~]
$ nano lru.c
(kali@JOHNPRAVEEN P)-[~]
$ gcc lru.c -o lru
(kali@JOHNPRAVEEN P)-[~]
$ ./lru
Enter Number of Pages: 2
Enter Reference String:
5
3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@JOHNPRAVEEN P)-[~]
$
```

SHELL PRROGRAMMING : LRU

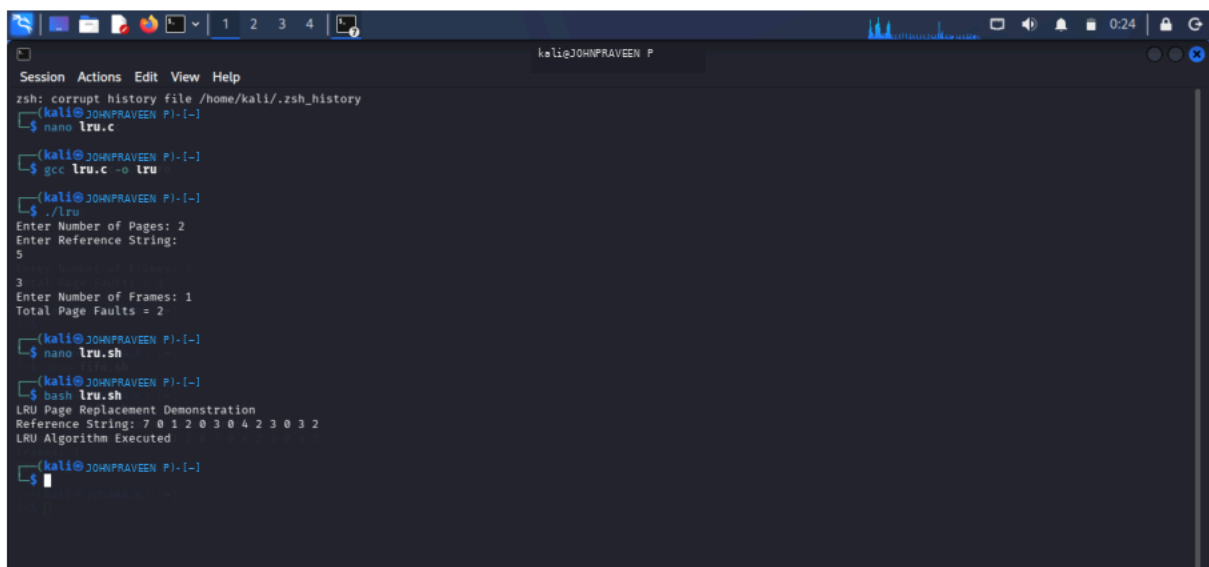


A screenshot of a terminal window with the nano 9.0 editor open. The editor is editing a file named `lru.sh`. The script content is as follows:

```
#!/bin/bash
echo "LRU Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "LRU Algorithm Executed"
```

The terminal window title is `kali@JOHNPRAVEEN P`. The nano editor's status bar at the bottom shows various keyboard shortcuts like `Ctrl+H` for Help, `Ctrl+W` for Write Out, etc.

OUTPUT :

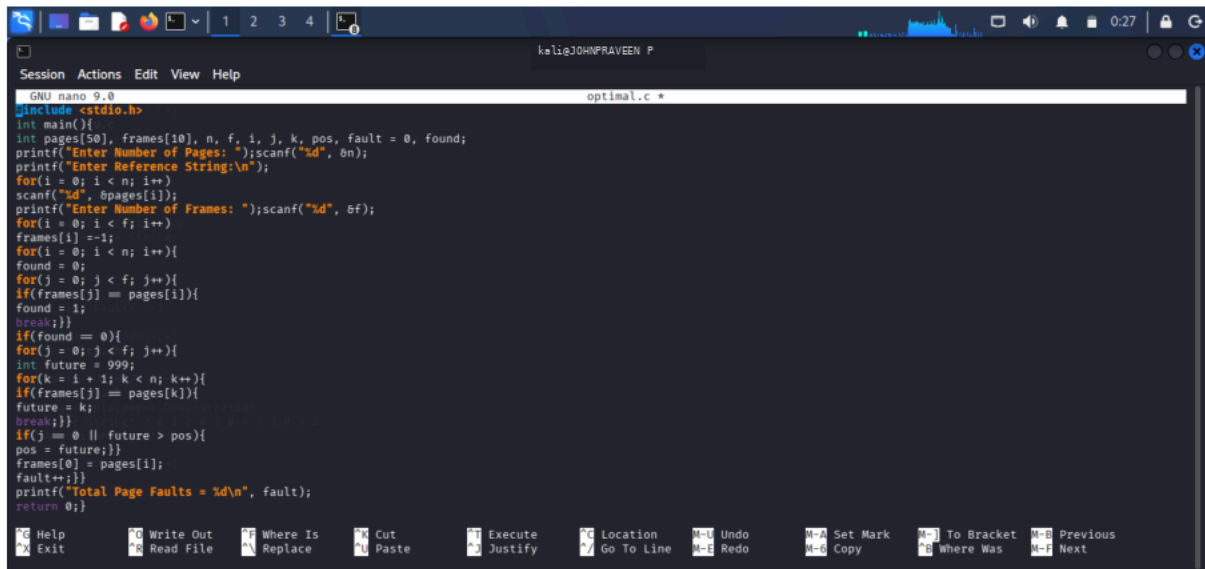


A screenshot of a terminal window showing the execution of the LRU program. The terminal output is as follows:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHNPRAVEEN P)-[~]
$ nano lru.c
(kali@JOHNPRAVEEN P)-[~]
$ gcc lru.c -o lru
(kali@JOHNPRAVEEN P)-[~]
$ ./lru
Enter Number of Pages: 2
Enter Reference String:
5
3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@JOHNPRAVEEN P)-[~]
$ nano lru.sh
(kali@JOHNPRAVEEN P)-[~]
$ bash lru.sh
LRU Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
LRU Algorithm Executed
(kali@JOHNPRAVEEN P)-[~]
$
```

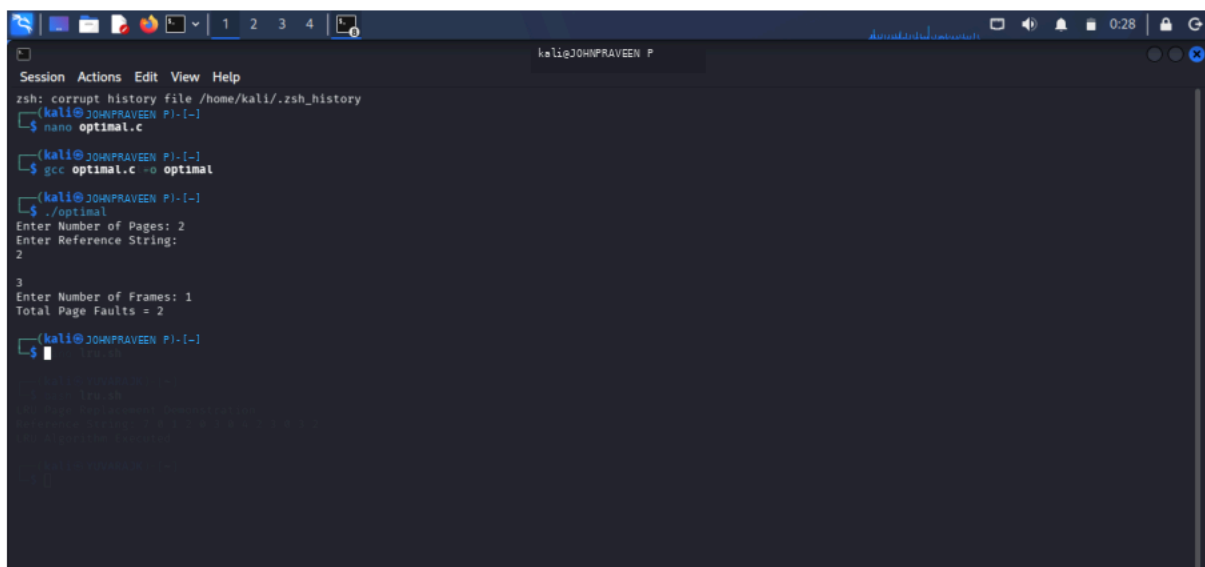
The terminal window title is `kali@JOHNPRAVEEN P`. The output shows the compilation of `lru.c` into `lru`, followed by the execution of `lru` and `lru.sh`, which displays the LRU Page Replacement Demonstration results.

C PROGRAMMING :OPTIMAL PAGE



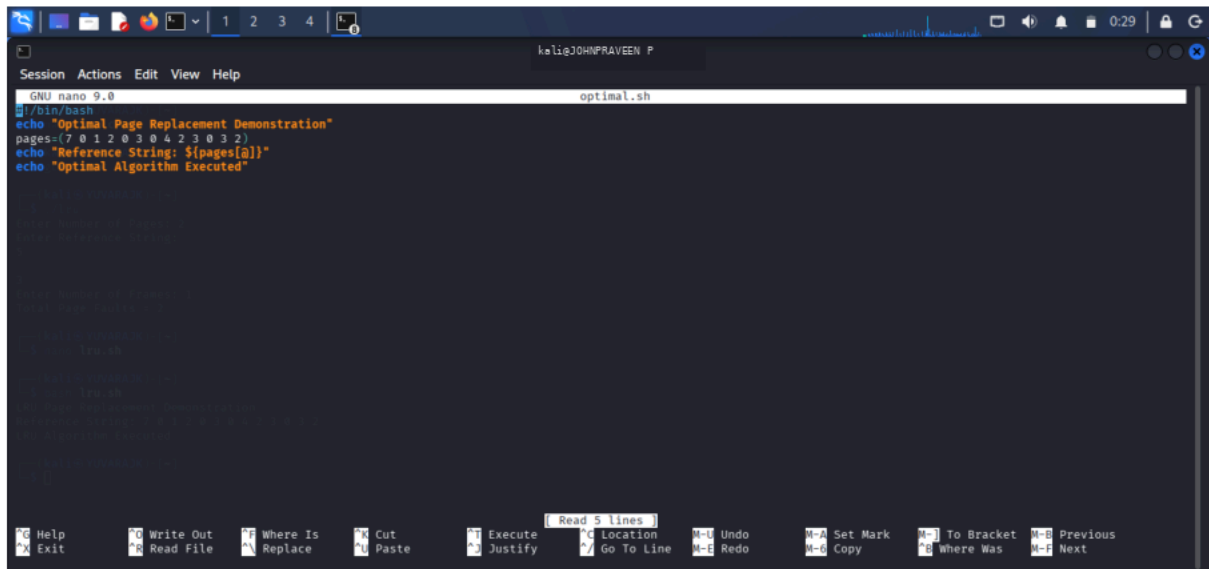
```
GNU nano 9.0 optimal.c *
#include <stdio.h>
int main(){
    int pages[50], frames[10], n, f, i, j, k, pos, fault = 0, found;
    printf("Enter Number of Pages: ");scanf("%d", &n);
    printf("Enter Reference String:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &pages[i]);
    }
    printf("Enter Number of Frames: ");scanf("%d", &f);
    for(i = 0; i < f; i++){
        frames[i] = -1;
    }
    for(i = 0; i < n; i++){
        found = 0;
        for(j = 0; j < f; j++){
            if(frames[j] == pages[i]){
                found = 1;
                break;
            }
        }
        if(found == 0){
            for(j = 0; j < f; j++){
                int future = 999;
                for(k = i + 1; k < n; k++){
                    if(frames[j] == pages[k]){
                        future = k;
                        break;
                    }
                }
                if(j == 0 || future > pos){
                    pos = future;
                }
            }
            frames[0] = pages[i];
            fault++;
        }
        printf("Total Page Faults = %d\n", fault);
    }
    return 0;
}
```

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@JOHNPRAVEEN P)-[~]
$ nano optimal.c
(kali@JOHNPRAVEEN P)-[~]
$ gcc optimal.c -o optimal
(kali@JOHNPRAVEEN P)-[~]
$ ./optimal
Enter Number of Pages: 2
Enter Reference String:
2
3
Enter Number of Frames: 1
Total Page Faults = 2
(kali@JOHNPRAVEEN P)-[~]
$
```

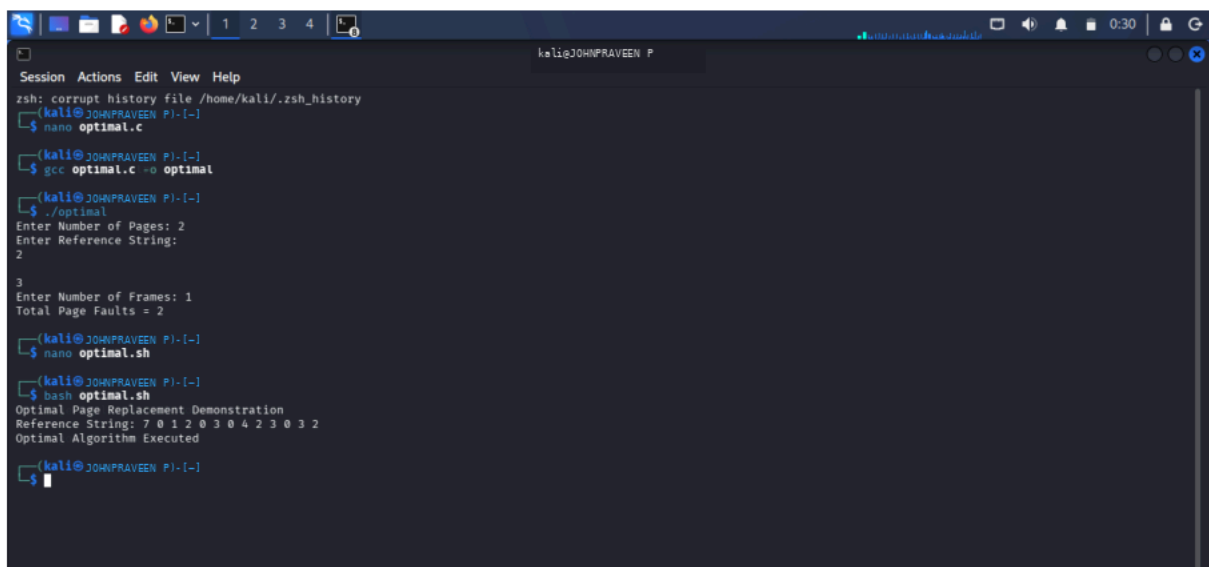
SHELL PROGRAMMING :



```
GNU nano 9.0 optimal.sh
#!/bin/bash
echo "Optimal Page Replacement Demonstration"
pages=(7 0 1 2 0 3 0 4 2 3 0 3 2)
echo "Reference String: ${pages[@]}"
echo "Optimal Algorithm Executed"
```

The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named 'optimal.sh'. The content of the file is a bash script that prints a title, an array of page numbers, and a reference string. The terminal window title is 'kali@JOHNPRAVEEN P' and the time is 0:29.

OUTPUT :



```
zsh: corrupt history file /home/kali/.zsh_history
kali@JOHNPRAVEEN P:~$ nano optimal.c
kali@JOHNPRAVEEN P:~$ gcc optimal.c -o optimal
kali@JOHNPRAVEEN P:~$ ./optimal
Enter Number of Pages: 2
Enter Reference String:
2
3
Enter Number of Frames: 1
Total Page Faults = 2
kali@JOHNPRAVEEN P:~$ nano optimal.sh
kali@JOHNPRAVEEN P:~$ bash optimal.sh
Optimal Page Replacement Demonstration
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2
Optimal Algorithm Executed
kali@JOHNPRAVEEN P:~$
```

The screenshot shows a terminal window where the user has compiled and executed the 'optimal' program. The program prompts for the number of pages (2) and a reference string (7 0 1 2 0 3 0 4 2 3 0 3 2). It then prompts for the number of frames (1) and displays the total page faults (2). The user then edits 'optimal.sh' and runs it, which outputs the same information as the compiled program. The terminal window title is 'kali@JOHNPRAVEEN P' and the time is 0:30.